FDTD XPU technology on Systems with Non Uniform Computer Memory (NUMA) Architecture

A. Lauer[1], W. Simon[1], A. Wien[1],
[1]IMST GmbH, D–47475 Kamp-Lintfort, Germany, e-mail lauer@imst.de

***Abstract*—3D EM simulations are nowadays widely used to design antenna systems, RF components and RF modules. The FDTD method shows significant advantages compared to frequency domain methods if the problems are getting complex and have to consider thousands of objects and complex multilayer circuits. This paper describes how the FDTD XPU technology (software accelerated computing on modern CPUs) can utilize the Non Uniform computer Memory Architecture (NUMA) to further accelerate FDTD simulations on modern workstations and servers. For small and large simulations this yields to much shorter simulation times when using the XPU FDTD technique on workstations compared to using hardware acceleration with GPUs.**

***Index Terms*—EM simulation, acceleration techniques, FDTD, NUMA, Computer Technology, GPU, Antennas, RF components.**

I. Introduction

The well known FDTD method [1], [2] has been recently [3], [4] reported to be enhanced with the XPU technology to use modern PC computer's CPU cache memory for ultra-fast simulation speed. On a Xeon E5 2690[1] based PC with two CPU modules approx. 5000 Million XPU Yee cells per second (MCells/s) speed have been reported in [3]. The newer Xeon E5 2697 V3[2] performs with up to approx. 8000 XPU MCells/s [4].

The publication at hand explains in detail, how the XPU technique is used for optimal performance on both machines. The exact time stepping scheme is explained under the aspect of the non uniform memory architecture (NUMA) of both computers[3]. XPU Performance measurements with an empty 100 MCell structure are investigated for both systems under different NUMA conditions.

II. The XPU FDTD stepping strategy

As reported in [3], [4], the XPU FDTD performance enhancement is based on the fact, that classical FDTD simulation speed is mainly limited by the performance of the computer's main memory interface. This bottleneck is overcome by keeping FDTD data belonging to multiple time steps in computer memory at the same time. If for every FDTD cell the next time step is done as soon as possible, in most cases the field data is read from/written to the computer's cache memory. The cache memory's bandwidth is a multiple of the bandwidth of the main memory.

Assumptions for the XPU multiple time stepping algorithm in focus here are [3]:

1) One FDTD cell consists of six field samples $E_x, E_y, E_z, H_x, H_y, H_z$.
   - E_x, E_y, E_z-Field components located in the middle of the x,y,z-min edges of the cell and
   - H_x, H_y, H_z-Field components located in the middle of the x,y,z-min faces of the cell.

   E and H Field of one cell can then be time-stepped together[4].
2) One complete x-Row of FDTD cells is time stepped one after another, so the actual time step number is regarded not to depend on x hereafter.
3) Equation (3) in [3] to analyze if time step $n \to n+1$ is permitted in the row indexed by (Y, Z) yields
   - Row $(Y+1, Z)$ has to be on time step n,
   - Row $(Y, Z+1)$ has to be on time step n,
   - Row $(Y, Z-1)$ has to be on time step $n+1$,
   - Row $(Y-1, Z)$ has to be on time step $n+1$.
4) Rows outside the field area (e.g. $Y < 0$) don't count in the conditions.

The XPU FDTD stepping strategy is now investigated assuming a simulation volume of $n_\mathrm{x} \times n_\mathrm{y} = 8 \times n_\mathrm{z} = 9$.

Fig. 1 sketches FDTD cell rows:

(a) shows the initial state: all rows are at $t = 0$. The stepping's geometry is sketched outside the simulation area. The rectangle on the right side will do the $0 \to 1$ time step, the second one the $1 \to 2$ time step and so on.
(b) shows the state after the first stepping rectangle has moved into the simulation area and the work is done. First

[1]Xeon E5 2690 is based on Intel's Sandy Bridge Microarchitecture, released 2012.

[2]Xeon E5 2697 V3 is based on Intel's Haswell Microarchitecture, released 2014.

[3]Each CPU Module has four Memory Channels (DDR3/DDR4). To access the other CPU's memory, data has to go through an QPI interface following a special Cache Coherence Protocol.

[4]first H, than E

Authorized licensed use limited to: OREGON STATE UNIV. Downloaded on February 24,2026 at 23:52:33 UTC from IEEE Xplore. Restrictions apply.

the $(Y = 0, Z = 0)$ row fulfills the stepping condition from $0 \rightarrow 1$, after that row $(Y = 0, Z = 1)$ does $0 \rightarrow 1$, up-to row $(Y = 0, Z = 3)$.

(b) shows the state after the first two stepping rectangles have moved into the simulation area and the work is done. Stepping conditions are OK to do $0 \rightarrow 1$ and $1 \rightarrow 2$ stepping show in the figure.

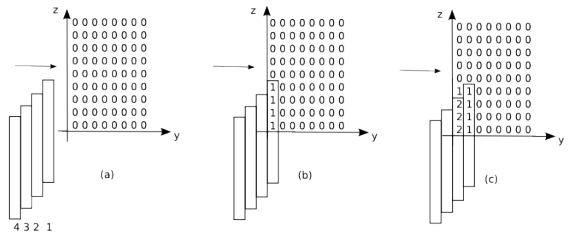

Fig. 1. Yee Cell x-rows at the beginning of the simulation. The numbers are the row's actual time step after time stepping. The rectangles indicate the XPU time-stepping's geometry with $N = 4$ multi stepping with a length of $M = 8$. The the size of the simulation area is $n_{\mathrm{y}} = 8 \times n_{\mathrm{z}} = 9$.

Fig. 2 sketches FDTD cell rows near to the end of one multi-time-step, the time stepping geometry is performing the second sweep, the offset in y direction to the first sweep in Fig. 1 is $M - N$ (M:length of multi-stepping block, N: multi stepping factor performing N time steps in parallel)

(a) shows the state after the time stepping geometry has moved through the structure for the first time,
(b) shows the beginning of the second sweep,
(c) shows the state in the middle of the second sweep.

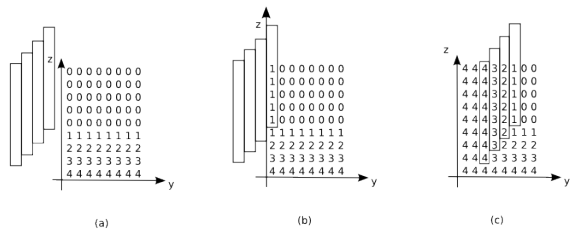

Fig. 2. Yee Cell x-rows during simulation. The numbers are the row's actual time step after time stepping.

Now the focus goes to more realistic simulation structures. An $n_{\mathrm{x}} = 700, n_{\mathrm{y}} = 200, n_{\mathrm{z}} = 700$ air filled FDTD structure is investigated, yielding approx. 100 Mio. FDTD Cells[5]. In this case, the shown (N, M) simulation strategy performs N time steps in

$$n_{sweep} = n_{\mathrm{y}} \times M \tag{1}$$

cell rows in one normal sweep[6].

[5]This means 2.35 GBytes of computer memory for field storage with single precision floating point numbers.

[6]The first and last sweep do less rows.

Assuming the CPU cache is big enough to hold approx.

$$n_{cache} = N \times M \tag{2}$$

rows,

$$n_{\mathrm{RAM}} = (N + M) \times n_{\mathrm{y}} \tag{3}$$

rows have to be loaded/stored from/to computer main memory for one sweep.

This reduces the main memory traffic by a factor η of approx.

$$\eta = N \times \frac{n_{sweep}}{n_{\mathrm{RAM}}} = \frac{N \times M}{N + M}. \tag{4}$$

Table I shows the simulation speed and the memory traffic reduction factors for the 100 Mio. cell structure on one Xeon E5 2697 V3 CPU module. Here N multi-steps are taken for each of the 14 CPU cores.

N	$N \times 14$	M	η	$n_{cache} \times 14$	M$Cells/s$	$\frac{\mathrm{M}Cells/s}{\eta}$
1	14	3	2.471	42	1613	653
2	28	6	4.941	168	3246	657
3	42	9	7.412	378	3914	528
4	56	12	9.882	672	4530	458
5	70	15	12.353	1050	4860	393
6	84	18	14.824	1512	4969	335
7	98	21	17.294	2058	4597	266

TABLE I
SIMULATION SPEED IN DEPENDENCE OF XPU PARAMETERS FOR AN EMPTY $n_{\mathrm{x}} = 700, n_{\mathrm{y}} = 200, n_{\mathrm{z}} = 700$ STRUCTURE ON ONE XEON E5 2697 V3 MODULE WITH 14 CORES.

The Xeon E5 2697 V3 L3 Cache size is 35 MBytes or approx. 2083 x-rows. This explains why the automatic (on-the-fly speed measurement based) XPU strategy optimization in EMPIRE XPU [5] uses $N = 6$ per CPU core and $M = 18$, which is the fastest option in table I. The last row shows the simulation speed projected to $\eta = 1$ in order to evaluate the memory speed.

The well known memory bandwidth measurement tool *STREAM* [6] reports 36.7 GBytes/s copy memory bandwidth on on Xeon E5 2697 V3 module on our system, giving a theoretical FDTD speed (48 Bytes R/W per cell) of 764 MCells/s at $\eta = 1$, if only memory bandwidth is the limiting factor. This agrees very well with the first two rows in table I. From row three on, the memory bandwidth is not the (only) limiting factor anymore.

III. THE XPU METHOD ON NUMA MACHINES

Both computers under investigation here (Xeon E5 2697 V3 , Xeon E5 2690) have two CPU modules. Four memory controllers are available per CPU module, this means the RAM memory is bound to its module. If the other CPU module wants to access it, data has to be passed through. The concept

Authorized licensed use limited to: OREGON STATE UNIV. Downloaded on February 24,2026 at 23:52:33 UTC from IEEE Xplore. Restrictions apply.

behind that is abstracted as *non uniform memory architecture* or NUMA, if programs or threads can be bound to a CPU module (so called *NUMA node* in this context). Also allocated RAM memory can be bound to a certain node by software.

Table II summarizes Memory Bandwidths measured again width STREAM under different NUMA conditions.

CPU	Program Node	Memory Node	Bandwidth in MBytes/s	FDTD Speed MCells/s
Xeon	0	0	25400	3014
E5 2690	0	1	15600	2517
	0	0,1	31384	2957
Xeon	0	0	36700	4927
E5 2697	0	1	6599	1518
V3	0	0,1	10948	2206

TABLE II
MEMORY BANDWIDTH MEASURED WITH STREAM UNDER NUMA CONDITIONS. MEMORY NODE 0,1 MEANS INTERLEAVING MEMORY. ONLY ONE CPU MODULE IS USED FOR CALCULATION

It can be clearly seen, that for the older Xeon E5 2690 simple NUMA interleaving[7], which can be configured in the BIOS yields good performance. The up to date Xeon E5 2697 V3 obviously needs special NUMA treatment for optimal FDTD performance.

For reasons of simplicity *two* NUMA nodes are assumed here. This is correct for dual Xeon E5 2690 and dual Xeon E5 2697 V3 workstations. The XPU scheme proposed here can extended to more nodes easily.

The XPU enhanced time stepping scheme described in section II is adopted to NUMA needs in the following way

- The Field Memory is split among the memory-nodes in the middle of the z - direction at $z = z_{\mathrm{mid}}$.
- Both nodes *share* the execution of a *single* XPU scheme in a way that
  - node 0 updates all x-rows with $z \leq z_{\mathrm{mid}}$,
  - node 1 updates all x-rows with $z > z_{\mathrm{mid}}$.
- The needed multi–thread and multi–core synchronism is maintained by use of a two dimensional integer number array holding the actual time step for each x-row.
- Synchronism is maintained by waiting for the proper time-steps to show up in the neighbor x-rows.

Table III shows the results for the empty test structure treated with the NUMA XPU scheme.

The Xeon E5 2697 V3 performs very well using the NUMA strategy described here. The Value of 8465 MCells/s is very close to $2 \times 4927 = 9854 MCells/s$, which corresponds to a very good parallelization factor of 0.86. The older Xeon E5 2690 is slightly better when using the interleaving mode.

[7]Interleaving means one cache line of 64 Bytes on Node 0, the next on Node 1, etc.

CPU	Mode	XPU Speed MCells/s
Xeon	XPU NUMA	5480
E5 2690	interleaved	5781
Xeon	XPU NUMA	8465
E5 2697	interleaved	3262

TABLE III
XPU SPEED ON DIFFERENT COMPUTERS USING DIFFERENT NUMA STRATEGIES.

IV. NUMA EXAMPLE: SATELLITE ANTENNA FOR SPACE BASED ADS-B

The XPU FDTD technique with NUMA support has been applied to the simulation of an installed satellite antenna. This satellite antenna operates at 1090 MHz and consists of four antenna panels:

- 3 panels with 5x3 antenna elements and
- 1 panel with 3x3 antenna elements.

All panels are mounted on an exemplary satellite to calculate realistic antenna beams. The installed satellite antenna together with a visualization of the electric field of one beam at 1090 MHz is shown in Fig. 3. The antenna design is described in more detail in [7]. The simulation model of the antenna and the satellite has a total size 232 x 611 x 321 FDTD cells (45 Million FDTD cells). This results in a grand total of 1.65 GB memory usage including near-field and far-field recording. The simulation performance when applying different strategies of the NUMA enhanced XPU FDTD technique to this application example are shown in table IV. The best strategy is automatically tested and chosen for every simulation with EMPIRE XPU.

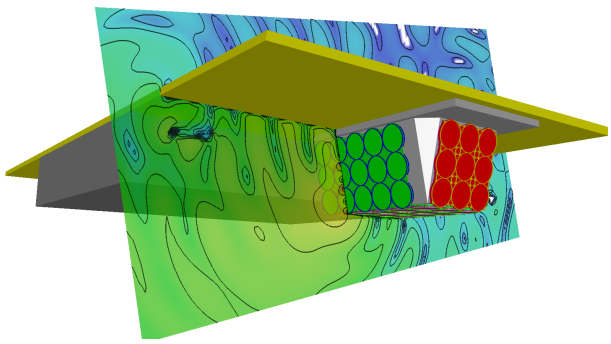

Fig. 3. Electric field at 1.090 GHz of the ADS-B antenna installed on an exemplary satellite platform (one beam only).

A maximum simulation speed of 5087 million cells/s is achieved on a dual Xeon E2697 workstation using all 28 CPU Cores with NUMA enhanced XPU FDTD technique. The total simulation time including near field and 3D far field calculation is approximately 8 minutes, making it perfectly

Authorized licensed use limited to: OREGON STATE UNIV. Downloaded on February 24,2026 at 23:52:33 UTC from IEEE Xplore. Restrictions apply.

Strategy	Multi factor N	XPU Speed in MCells/s
0	1	794
	2	3124
1	2	1255
	3	2421
	..	...
	6	2075
	7	2071
2	1	2636
	2	3419
	..	...
	8	5012
3	1	2386
	2	3465
	3	3949
	..	...
	8	5087
	9	4965

TABLE IV

SIMULATION SPEEDS WHEN APPLYING DIFFERENT STRATEGIES OF THE XPU FDTD TECHNIQUE TO THE SATELLITE ANTENNA SIMULATION ON A DUAL XEON E5 2697 V3 WORKSTATION. THE STRATEGIES 2 AND 3 CORRESPOND TO THE METHOD DESCRIBED HERE (M IS CALCULATED AUTOMATICALLY). STRATEGY 0 IS FOR SMALL STRUCTURES, STRATEGY 1 FOR SMALL TO MEDIUM SIZE ARRANGEMENTS. THE MODEL INCLUDES LOSSY MATERIALS AND EXTENSIVE FIELD RECORDING.

possible to perform e.g. detail optimization on the complete satellite model.

V. CONCLUSION

The extension of the XPU FDTD technique with Non Uniform Memory Access (NUMA) support is explained in detail. It has been demonstrated that the FDTD calculation speed of EMPIRE XPU is strongly increased when using FDTD XPU technology with NUMA, especially when modern multi core CPUs and multi CPU workstations are used. A simulation speed of over 8000 million FDTD cells per second is achieved on a standard Xeon E5 2697 V3 two CPU server computer.

REFERENCES

[1] K. S. Yee, "Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media," *IEEE Trans. Antennas Propagat.*, vol. 14, pp. 302–307, Mar. 1966.

[2] A. Lauer and I. Wolff, "Stable and efficient ABCs for graded mesh FDTD," in *IEEE MTT-S Int. Microwave Symp.*, vol. 2, (Baltimore, MD), pp. 461–464, June 1998.

[3] A. Lauer, W. Simon, and A. Wien, "XPU technology for fast and efficient FDTD simulations using modern cpus cache memory bandwidth," in *IEEE Conference Publications, Antennas and Propagation (EuCAP), 2015 9th European Conference on*, pp. 1–4, 2015.

[4] W. Simon, A. Lauer, and R. Baggen, "Designing complex frontend modules for ka-band and wigig communication using XPU-technology," in *APCAP Conference 2015*, 2015.

[5] A. Wien, "EMPIRE XPU Manual," 2015. [Online; http://www.empire.de/EMPIRE-Manual.pdf.

[6] P. John D. McCalpin, "Stream: Sustainable memory bandwidth in high performance computers," 2015. [Online; accessed 02-Oct-2015].

[7] W. Simon, R. Baggen, and A. Lauer, "EMPIRE XCcel: Efficient simulation of RF MEMS and antennas with XPU FDTD technology," in *Antennas and Propagation Society International Symposium (APSURSI), 2013 IEEE*, pp. 2327–2328, July 2013.

Authorized licensed use limited to: OREGON STATE UNIV. Downloaded on February 24,2026 at 23:52:33 UTC from IEEE Xplore. Restrictions apply.